



**Universidad
Tecnológica
del Perú**

- **Inicio**

Diseño e implementación de Arquitectura Empresarial

Proyectos de Desarrollo de Software:
Semana 01



**Universidad
Tecnológica
del Perú**

Revisamos el sílabo

- Dentro de este documento encontraremos información sobre el curso, incluyendo:
 - Logro del curso.
 - Cronograma de actividades.
 - Sistema de evaluación.
 - Bibliografía.





 TEMA ACTUAL

Proyectos de Desarrollo de Software

Gestión de Proyectos

Buenas prácticas y errores comunes en el ciclo de vida del software.

Gestión de Riesgos

Identificación, análisis y mitigación de riesgos en proyectos de software.

Modelos y Metodologías

Proceso de software, RUP y las 6 mejores prácticas de desarrollo.



🎯 LOGRO DE APRENDIZAJE

¿Qué lograrás hoy?

Al finalizar la sesión, el estudiante será capaz de analizar y aplicar los principios de gestión de proyectos de software, modelos de proceso y metodologías como RUP, identificando buenas prácticas y riesgos comunes, mediante el estudio de casos reales y marcos de referencia actuales.

¿Para qué te sirve esto?



En el trabajo

Liderar o participar en proyectos de software con criterio profesional y metodológico.



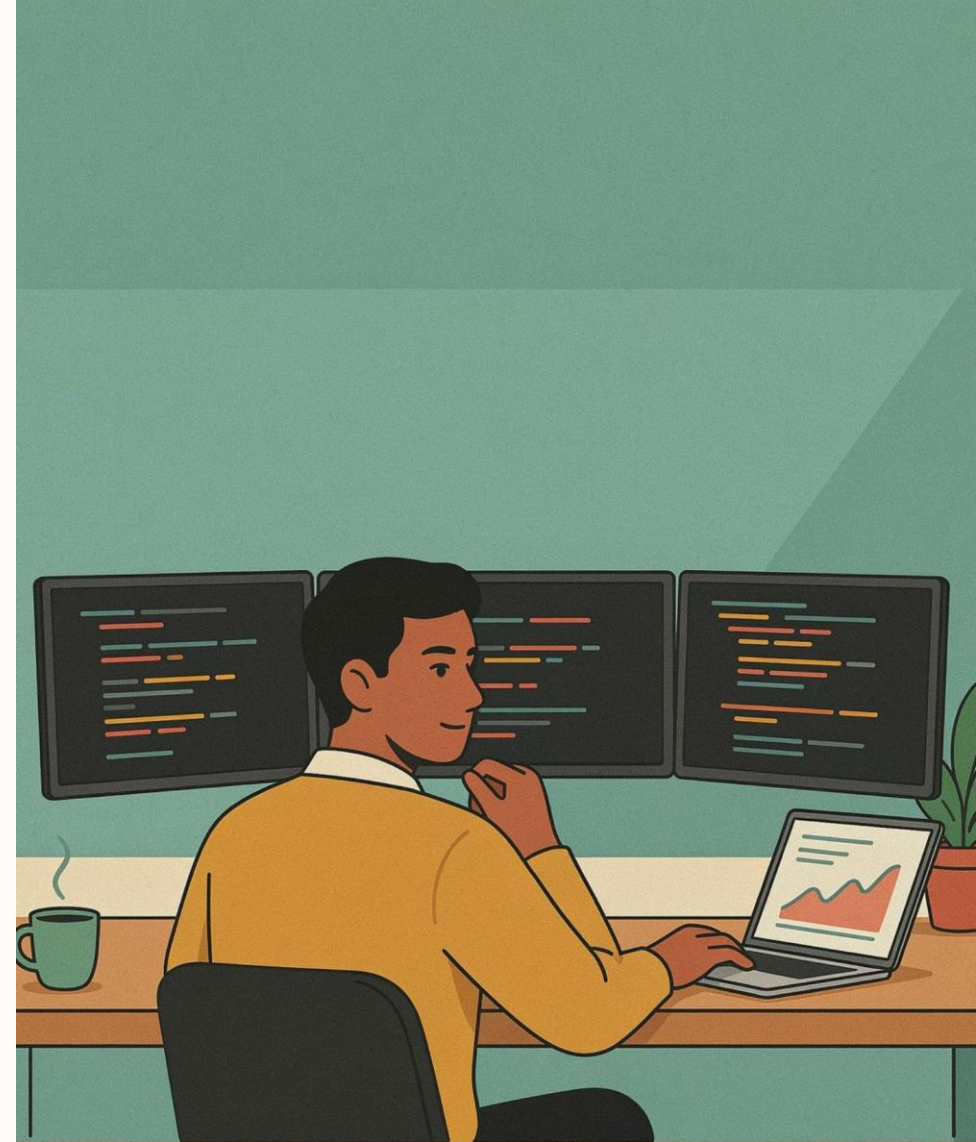
En la gestión

Anticipar riesgos, evitar errores costosos y tomar decisiones informadas.



En tu carrera

Dominar RUP y metodologías ágiles es una competencia altamente valorada en el mercado laboral.



 MOTIVACIÓN

¿Por qué importa la gestión de proyectos?

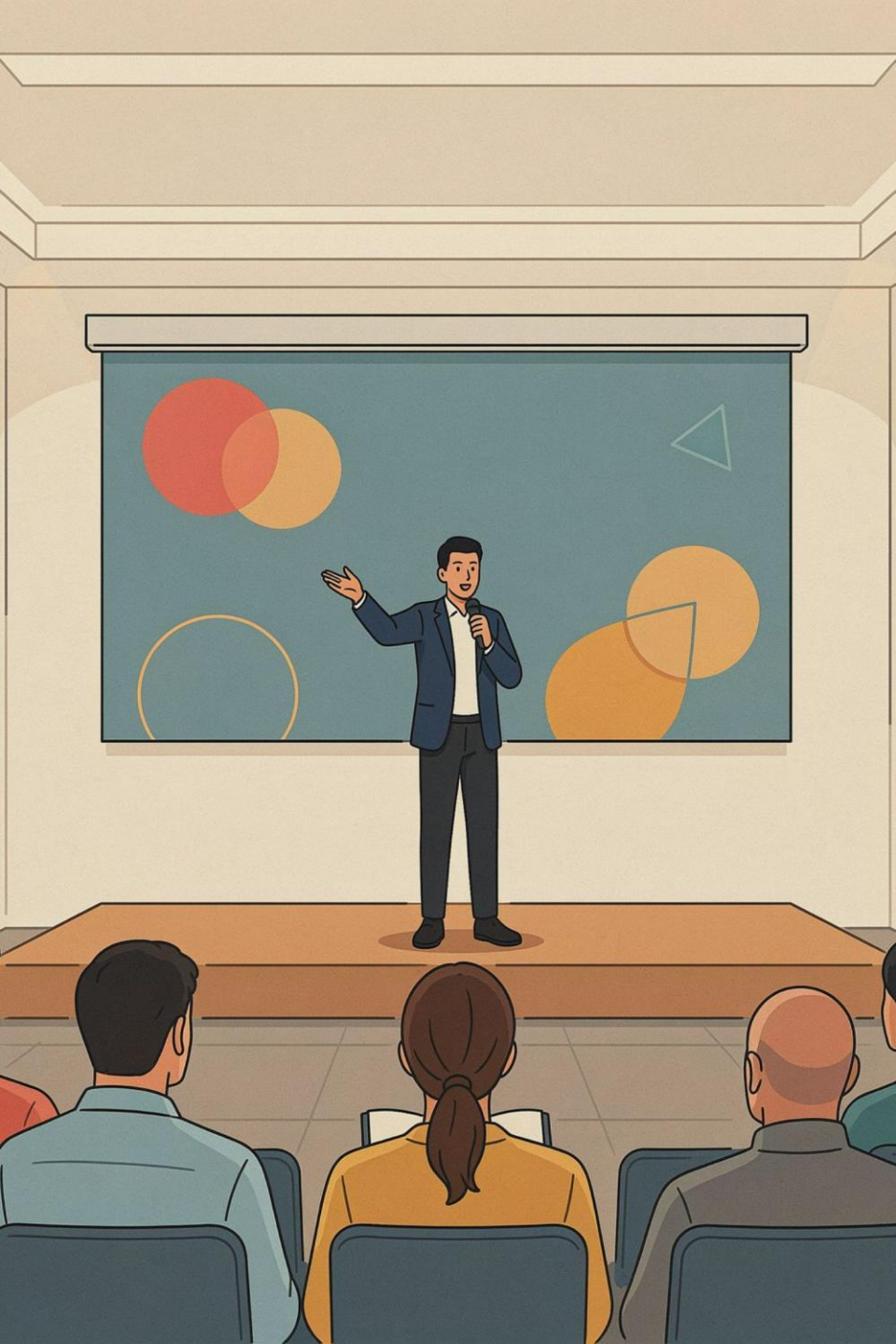
"El 70% de los proyectos de software fracasan por mala gestión, no por falta de talento técnico." — Standish Group, CHAOS Report

Mira este video introductorio sobre gestión de proyectos de software en español y reflexiona: ¿cuántos de estos errores has visto o podrías cometer?



 Video recomendado: «Gestión de Proyectos de Software» — busca en YouTube: "*Gestión de proyectos de software explicado fácil*" (canal Platzi o Fazt Code, duración ~10 min)

<https://www.youtube.com/watch?v=TLVDBAo1aEY>



 CONOCIMIENTOS PREVIOS

Activación Cognitiva

Antes de comenzar, reflexiona sobre estas preguntas. No hay respuestas incorrectas — el objetivo es activar lo que ya sabes:

1

¿Qué es un proyecto de software?

¿En qué se diferencia de un proyecto de construcción o de manufactura?

2

¿Has participado en un proyecto?

¿Qué salió bien? ¿Qué falló? ¿Hubo una metodología definida?

3

¿Qué es un riesgo en software?

¿Cómo lo identificarías antes de que se convierta en un problema real?

4

¿Conoces RUP o alguna metodología ágil?

¿Scrum, Kanban, XP? ¿Cuál usarías y por qué?

Gestión de Proyectos de Software

Un proyecto de software es un esfuerzo temporal con inicio, fin y objetivo definido: entregar un producto de software funcional. La gestión implica planificar, ejecutar, monitorear y cerrar ese esfuerzo con eficiencia.



☑ BUENAS PRÁCTICAS

Buenas Prácticas en Gestión de Proyectos

Definir el alcance claramente

Documentar qué incluye y qué no incluye el proyecto desde el inicio evita el temido *scope creep*.

Comunicación continua

Reuniones regulares con el equipo y los stakeholders garantizan alineación y detección temprana de problemas.

Gestión del cambio

Todo cambio de requerimientos debe pasar por un proceso formal de evaluación de impacto antes de aprobarse.

Métricas y seguimiento

Usar KPIs como velocidad del equipo, deuda técnica y cobertura de pruebas para tomar decisiones basadas en datos.

▲ ERRORES COMUNES

Errores Frecuentes que Arruinan Proyectos

Requerimientos vagos

Iniciar sin especificaciones claras genera retrabajo y conflictos con el cliente.

Subestimar el tiempo

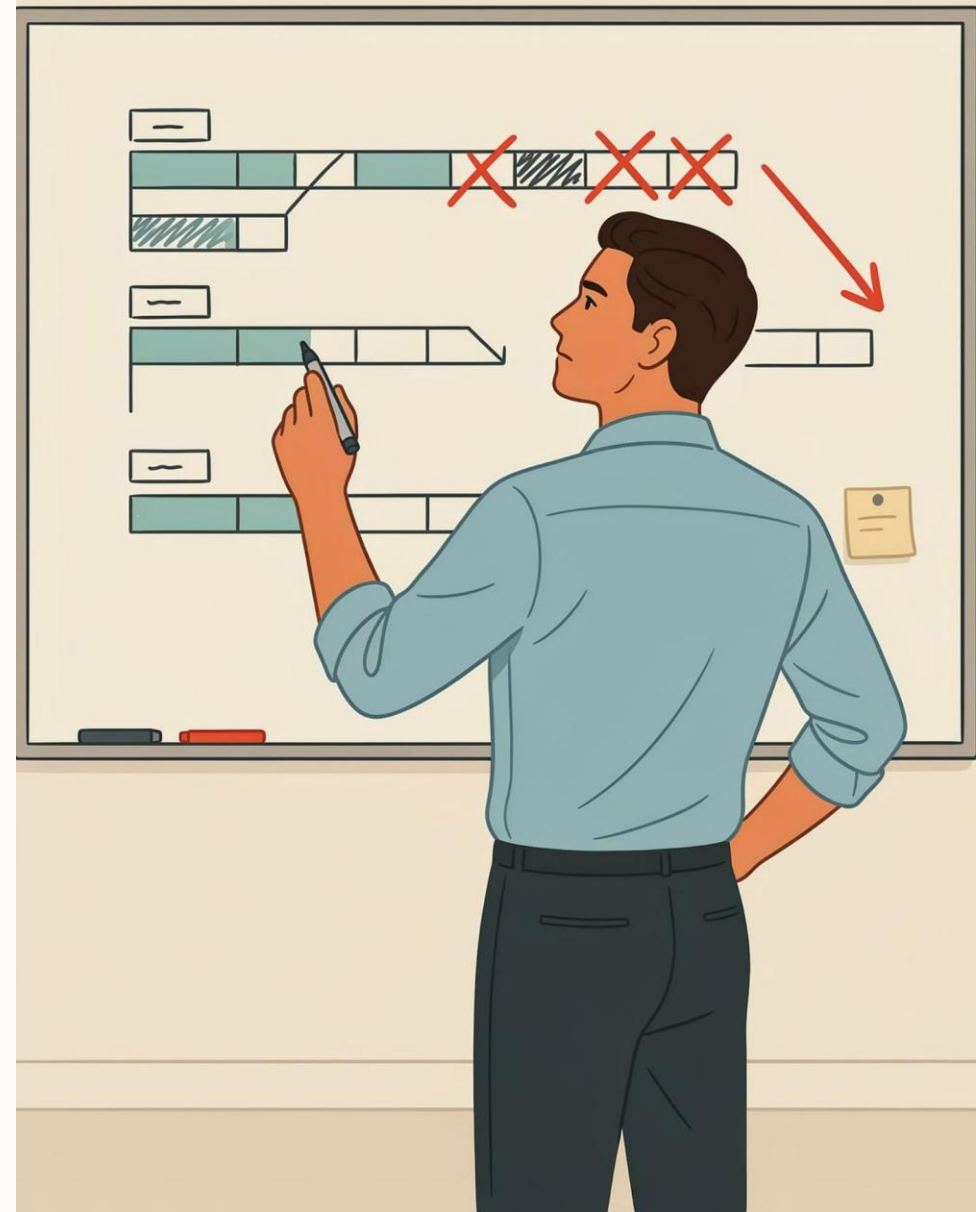
El optimismo sin datos históricos lleva a cronogramas irreales y entregas tardías.

Ignorar la deuda técnica

Acumular código de baja calidad para cumplir plazos destruye la mantenibilidad futura.

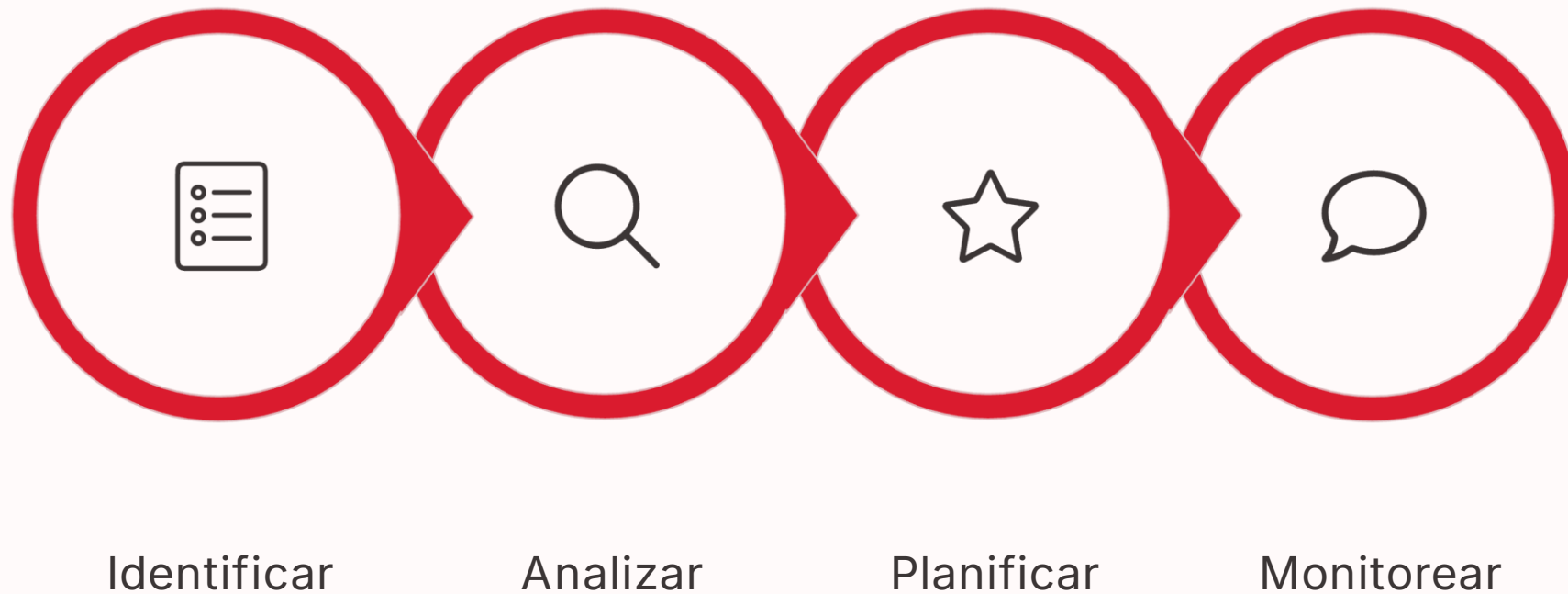
Falta de pruebas

No probar continuamente resulta en defectos costosos descubiertos en producción.



Gestión de Riesgos en Software

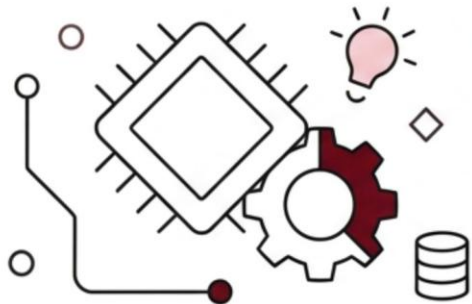
Un riesgo es un evento incierto que, si ocurre, impacta negativamente el proyecto. Gestionarlo proactivamente es la diferencia entre un proyecto exitoso y uno fallido.



La gestión de riesgos no es un evento único — es un proceso continuo durante todo el ciclo de vida del proyecto.

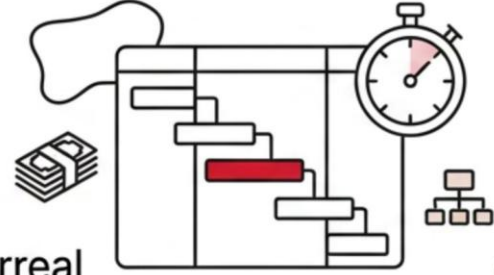
Tipos de Riesgos en Proyectos de Software

RIESGOS TÉCNICOS



- Tecnología nueva
- Integración compleja
- Deuda técnica

RIESGOS DE GESTIÓN



- Alcance mal definido
- Recursos insuficientes
- Cronograma irreal

RIESGOS EXTERNOS



- Cambios regulatorios
- Dependencias de terceros
- Cambios de mercado

RIESGOS DE EQUIPO



- Rotación de personal
- Falta de habilidades
- Conflictos internos

Identificar a qué categoría pertenece cada riesgo permite asignar al responsable correcto y aplicar la estrategia de respuesta más adecuada.

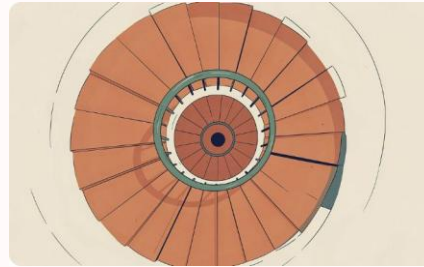
Modelos del Proceso de Software

Un modelo de proceso define el marco de trabajo que guía cómo se desarrolla el software. Cada modelo tiene su contexto ideal de aplicación.



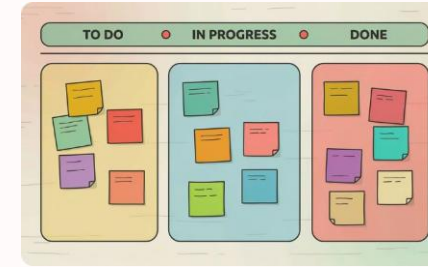
Cascada (Waterfall)

Fases secuenciales y rígidas. Ideal para proyectos con requerimientos estables y bien definidos desde el inicio.



Espiral

Combina iteración con análisis de riesgos. Adecuado para proyectos grandes y complejos con alta incertidumbre.



Ágil / Iterativo

Entregas incrementales y adaptación continua. Ideal cuando los requerimientos evolucionan durante el proyecto.

Metodologías de Desarrollo de Software

Scrum

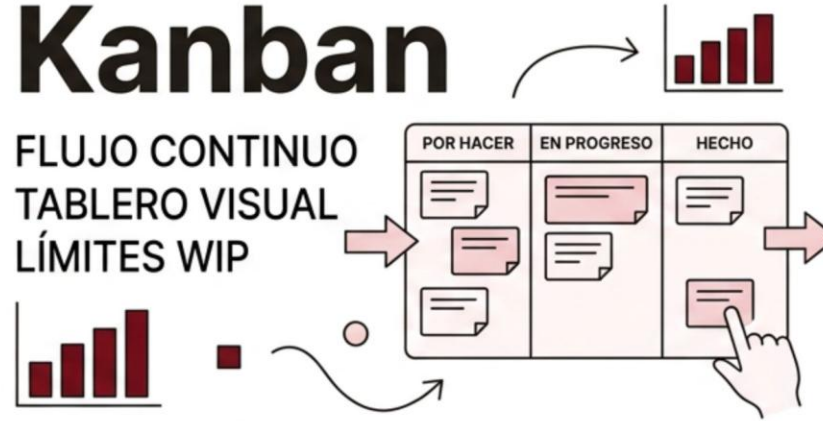
SPRINTS (2-4 sem)
ROLES DEFINIDOS
ARTEFACTOS

Scrum Master
Product Owner



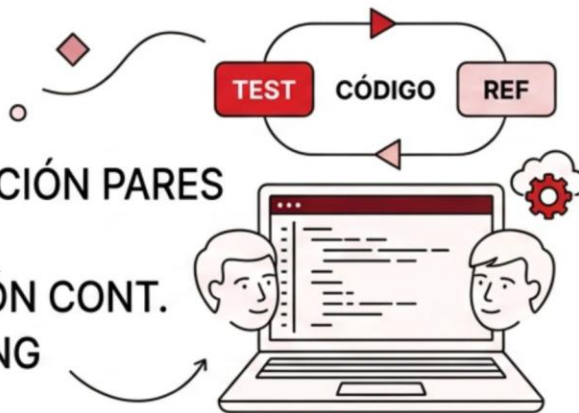
Kanban

FLUJO CONTINUO
TABLERO VISUAL
LÍMITES WIP



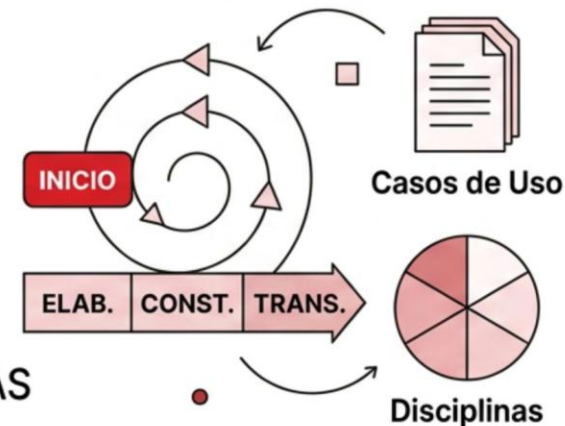
XP

PROGRAMACIÓN PARES
TDD
INTEGRACIÓN CONT.
REFACTORING



RUP

ITERATIVO
BASADO EN
CASOS USO
4 FASES
6 DISCIPLINAS



Conceptos Básicos del RUP

RUP (Rational Unified Process) es un proceso de desarrollo de software iterativo e incremental, desarrollado por Rational Software (IBM). Se basa en tres conceptos clave:

Dirigido por Casos de Uso

Los requerimientos se capturan como casos de uso que guían todo el desarrollo.

Centrado en la Arquitectura

La arquitectura del sistema es el eje central que organiza el diseño y la implementación.

Iterativo e Incremental

El software se construye en ciclos cortos, entregando valor en cada iteración.



Las 4 Fases del RUP

1

Inicio

Define el alcance, viabilidad y casos de negocio del proyecto.

2

Elaboración

Establece la arquitectura base y mitiga los riesgos técnicos principales.

3

Construcción

Desarrolla el sistema completo de forma iterativa e incremental.

4

Transición

Entrega el producto al usuario final, incluyendo pruebas y capacitación.

 Cada fase contiene una o más iteraciones. El esfuerzo varía por fase: la Construcción consume el mayor porcentaje del tiempo total del proyecto.

Disciplinas y Workflows del RUP

Disciplinas de Ingeniería

- Modelado del negocio
- Requerimientos
- Análisis y Diseño
- Implementación
- Pruebas
- Despliegue

Disciplinas de Soporte

- Gestión de configuración y cambios
- Gestión del proyecto
- Entorno (herramientas y procesos)

Los workflows describen las actividades, roles y artefactos de cada disciplina en cada fase. No todas las disciplinas tienen el mismo peso en todas las fases.

Las 6 Mejores Prácticas de Desarrollo de Software

1

Desarrollo iterativo

Construir el software en ciclos cortos para obtener retroalimentación temprana.

2

Gestión de requerimientos

Capturar, organizar y rastrear los requerimientos a lo largo del proyecto.

3

Arquitecturas basadas en componentes

Diseñar sistemas modulares, reutilizables y fáciles de mantener.

4

Modelado visual (UML)

Usar diagramas para comunicar el diseño de forma clara y sin ambigüedades.

5

Verificación continua de la calidad

Integrar pruebas en cada iteración, no solo al final del proyecto.

6

Control de cambios

Gestionar los cambios de forma sistemática para mantener la integridad del producto.

 PRÁCTICA

Caso Práctico: Sistema de Gestión Académica

Escenario: Una universidad contrata a tu equipo para desarrollar un sistema de gestión académica. El cliente tiene ideas vagas, el presupuesto es limitado y el plazo es de 6 meses.

01

Paso 1 — Definir el alcance

Reunión con stakeholders para identificar módulos: matrícula, notas, horarios y reportes. Se documenta el alcance y se firma.

03

Paso 3 — Seleccionar metodología

Se elige RUP adaptado con sprints de 3 semanas. Se definen roles: Arquitecto, Analista, Desarrolladores, QA.

02

Paso 2 — Identificar riesgos

Riesgo crítico: integración con sistema legado de pagos. Plan: asignar un sprint completo a la integración en la fase de Elaboración.

04

Paso 4 — Ejecutar y monitorear

Se entregan incrementos funcionales cada 3 semanas. El cliente valida y retroalimenta. Los riesgos se revisan en cada iteración.

Resultado del Caso Práctico

Entrega exitosa

El sistema se entregó en semana 24, con todos los módulos funcionales y validados por el cliente.

Riesgo mitigado

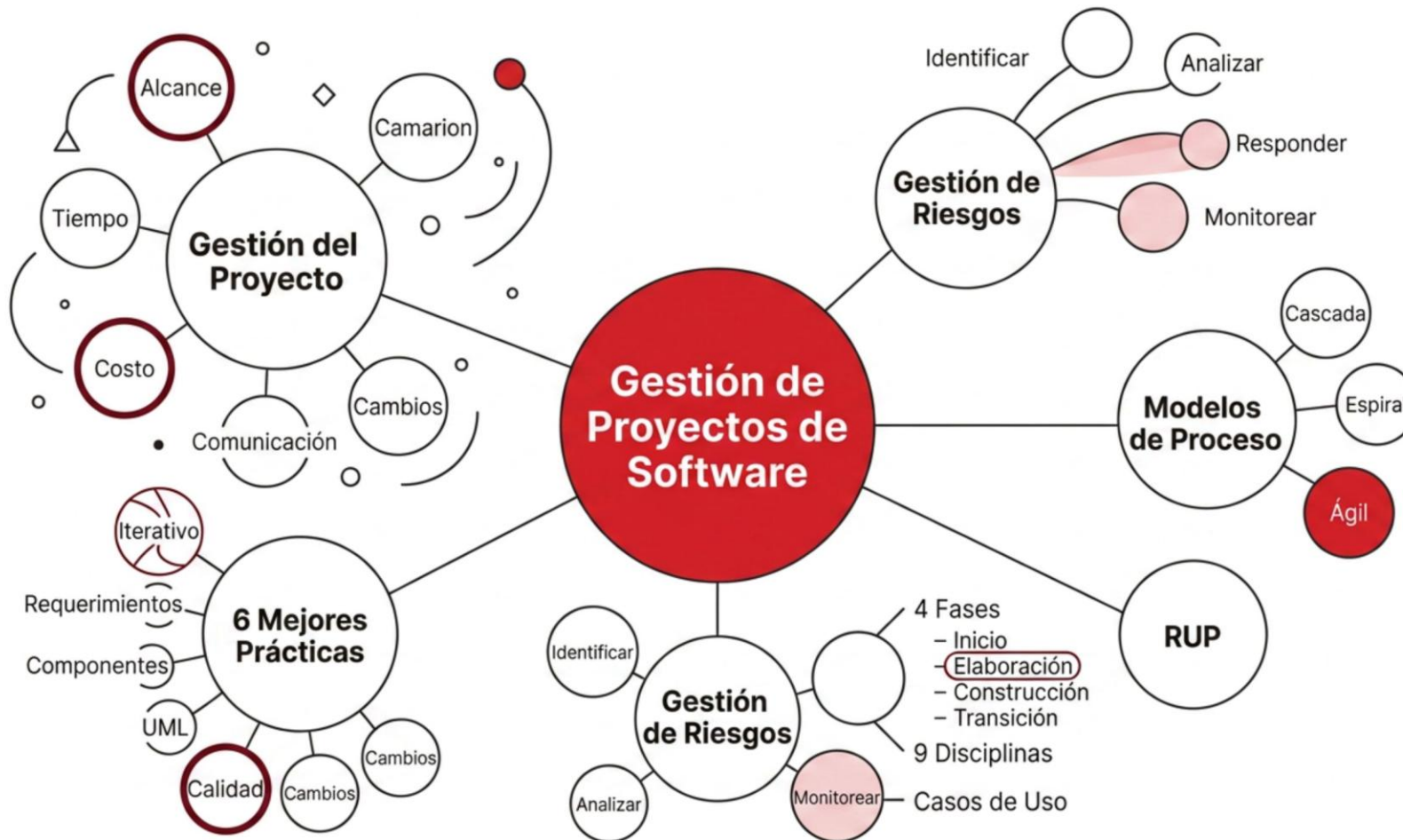
La integración con el sistema legado tomó 2 sprints extra, pero fue anticipada en la planificación.

Lección aprendida

La gestión proactiva de riesgos y el alcance bien definido fueron los factores clave del éxito.



Síntesis Visual del Tema



CONCLUSIONES

Ideas Clave de la Sesión

- La gestión es tan crítica como el código
La mayoría de los fracasos en software son de gestión, no técnicos.
Planificar bien es desarrollar bien.
- Los riesgos no gestionados se convierten en problemas
Identificar y planificar respuestas a los riesgos desde el inicio salva proyectos y presupuestos.
- RUP ofrece un marco robusto y adaptable
Sus 4 fases, 9 disciplinas y enfoque iterativo lo hacen aplicable a proyectos empresariales complejos.
- Las 6 mejores prácticas son universales
Independientemente de la metodología elegida, estas prácticas elevan la calidad de cualquier proyecto.





¿Qué hemos aprendido hoy?

Interactúa con estas preguntas para verificar tu comprensión:

¿Cuál es la diferencia entre un riesgo y un problema?

Pista: uno es potencial, el otro ya ocurrió. ¿Cómo cambiaría tu respuesta ante cada uno?

¿En qué fase del RUP se define la arquitectura base?

¿Por qué es importante hacerlo antes de la construcción masiva del sistema?

¿Qué práctica de las 6 aplicarías primero en un proyecto nuevo?

Justifica tu respuesta con un ejemplo concreto de tu contexto profesional o académico.

¿Qué metodología elegirías para el caso práctico de hoy y por qué?

¿Cambiarías algo del proceso que seguimos en el caso de la universidad?

 BIBLIOGRAFÍA

Referencias Académicas

Pressman, R. S. & Maxim, B. R. (2021)

Ingeniería del Software: Un enfoque práctico (9.^a ed.). McGraw-Hill Education.

Sommerville, I. (2016)

Software Engineering (10.^a ed.). Pearson Education Limited.

Standish Group (2020)

CHAOS Report 2020: Beyond Infinity. The Standish Group International. Recuperado de <https://www.standishgroup.com>

Kruchten, P. (2004)

The Rational Unified Process: An Introduction (3.^a ed.). Addison-Wesley Professional.

PMI — Project Management Institute (2021)

A Guide to the Project Management Body of Knowledge (PMBOK® Guide) (7.^a ed.). Project Management Institute.

Beck, K. et al. (2001)

Manifesto for Agile Software Development. Agile Alliance. Recuperado de <https://agilemanifesto.org>



**Universidad
Tecnológica
del Perú**



**Universidad
Tecnológica
del Perú**